Matrix Multiplication on the GPU with CUDA: From Elementwise Kernels to a Flattened Custom Kernel

Yashovardhan

October 3, 2026

Abstract

This report documents a small set of CUDA experiments, progressing from basic GPU kernels (a "hello world" launch, vector addition, elementwise multiplication) to two implementations of dense matrix multiplication: a naive version built on top of the elementwise kernels, and an optimized single-kernel version that maps each output element to one GPU thread using an explicit, hand-derived flattening of the 2D matrix indices into a 1D memory layout. The mathematical derivation behind the flattening and the operation-count argument that motivates a GPU implementation in the first place are presented, followed by the corresponding CUDA/PyCUDA code and benchmark results comparing both GPU kernels against NumPy on the CPU.

1 Motivation

Dense matrix multiplication $C = AB$ is the canonical example used to introduce GPU programming because it is both embarrassingly parallel across output elements and has a well-defined, regular memory access pattern. The notebook accompanying this report (run on a Google Colab instance with an NVIDIA T4 GPU) works through this example in three stages:

1. Sanity-check kernels to confirm the CUDA toolchain (`nvcc`) and the PyCUDA bridge from Python both work.
2. A naive matrix multiplication that reuses two very simple kernels (elementwise product and a trivial reduction) one matrix element at a time.
3. A single fused kernel, `customMatrixMul`, that computes every output element in parallel and is built directly from the index-flattening derivation in Section 2.

2 Mathematical Formulation

2.1 Operation count per output element

For matrices $A \in \mathbb{R}^{n\times j}$ and $B \in \mathbb{R}^{j\times m}$, the product $C = AB \in \mathbb{R}^{n\times m}$ is defined elementwise by

$$C_{ik} = \sum_{l=1}^{j} A_{il}\, B_{lk}. \tag{1}$$

Computing a single entry C_{ik} requires j multiplications and $j-1$ additions, i.e.

$$\text{cost}(C_{ik}) = 2j - 1 \text{ operations}. \tag{2}$$

Since there are nm independent output entries, the total operation count for the full product is

$$\text{cost}(C) = nm(2j-1) \text{ operations}. \tag{3}$$

Equation (3) is the starting point for a parallel decomposition: in the finest-grained scheme, one could in principle assign *nm blocks* (one per output element) with $2j-1$ *threads* per block, each thread handling one multiplication or one addition in the sum, so that the $nm(2j-1)$ scalar operations are spread across $nm(2j-1)$ threads total. In practice, the implementation in Section 5 uses a coarser, more practical decomposition: one thread per output element (nm threads total), with the inner sum over l left as a short sequential loop of length j inside each thread. This is exactly the point the derivation highlights with the annotation "loop $\rightarrow$ instead we use threads": the loop over output indices (i, k), which would be sequential on a CPU, is replaced by the GPU's thread grid, while only the reduction over the shared dimension j remains a loop.

2.2 Flattening a 2D index into 1D memory

A CUDA kernel receives matrices as flat (1D) device pointers, so the 2D index (i, j) of an entry A_{ij} of an m-row matrix must be mapped to a single linear offset k. Writing $A_{ij} \equiv a'_k$, the mapping used throughout is

$$k = m \cdot j + i, \tag{4}$$

i.e. the *column-major* (Fortran-order) convention: consecutive memory addresses walk down a column first (i varies fastest), then move to the next column (j increments). Equivalently,

$$a'[k] = a[i][j] = a'[mj + i]. \tag{5}$$

For a concrete 2×2 example with $m = 2$,

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \longrightarrow a' = \begin{bmatrix} a_{11} & a_{21} & a_{12} & a_{22} \end{bmatrix}, \tag{6}$$

since $a_{11} \rightarrow k = 0$, $a_{21} \rightarrow k = 1$, $a_{12} \rightarrow k = 2$, $a_{22} \rightarrow k = 3$, matching $k = mj + i$ with $m = 2$. The inverse (reconstruction) map recovers the 2D index from the flat one by $i = k \bmod m$, $j = \lfloor k/m \rfloor$.

2.3 Flattened form of the matrix product

Applying the flattening rule (4) to every matrix in Eq. (1) (with A having m rows, so $A_{jl} \rightarrow A[l \cdot m + i]$ for row index i, and similarly for B and C with k indexing the output column) gives the flat-array form actually implemented in the kernel:

$$C[k \cdot m + i] \;=\; \sum_{l} A[l \cdot m + i] \;\cdot\; B[k \cdot m + l]. \tag{7}$$

Equation (7) is a direct, line-for-line translation of Eq. (1) into the memory layout the GPU actually operates on, and it is exactly the expression implemented in the `customMatrixMul` kernel below.

3 GPU Programming Building Blocks

Before writing the matrix multiplication kernel, three minimal kernels were used to validate the toolchain and the basic thread-per-element pattern.

3.1 Device sanity check

A trivial kernel confirms that `nvcc` compiles correctly and that the device is reachable from a single thread:

```
__global__ void hello(){
  printf("Hello GPU");
}
int main(){
  hello<<<1,1>>>();
  cudaDeviceSynchronize();
  return 0;
}
```

3.2 Vector addition

Each thread handles one element of the output vector, with the thread index read directly from `threadIdx.x`:

```
__global__ void vecAdd(float* A, float* B, float* C){
  int i = threadIdx.x;
  C[i] = B[i] + A[i];
}
```

Memory for A, B, C is allocated with `cudaMallocManaged`, which provides a single unified pointer usable from both host and device, avoiding explicit `cudaMemcpy` calls for this simple case.

3.3 Elementwise product

Structurally identical to `vecAdd`, but multiplying instead of adding:

```
__global__ void dotProd(float *A, float *B, float *C){
  int i = threadIdx.x;
  C[i] = A[i] * B[i];
}
```

This kernel produces the elementwise product $A_i B_i$, not a scalar dot product; the reduction (summation) is deliberately left to the host, since combining the two kernels is the basis of the naive matrix multiplication in Section 4.

4 Naive Matrix Multiplication

The first matrix multiplication strategy reuses `dotProd` directly: for every output entry C_{ik}, row i of A and column k of B are copied to the GPU, multiplied elementwise with `dotProd`, copied back to the host, and summed with `np.sum`:

```
for i in range(M):
    for j in range(N):
        row_A_gpu = cuda.to_device(A[i, :])
        col_B_gpu = cuda.to_device(B[:, j])
        dot_prod_kernel(row_A_gpu, col_B_gpu, temp_mult_gpu,
                        block=(K, 1, 1), grid=(1, 1))
        temp_mult = np.empty(K, dtype=np.float32)
        cuda.memcpy_dtoh(temp_mult, temp_mult_gpu)
        C[i, j] = float(np.sum(temp_mult))
```

This is a direct, unoptimized translation of Eq. (1): the double loop over (i, k) runs on the host, and only the innermost sum over l is offloaded to the GPU, with a kernel launch and a host↔device transfer for every single output element. The result was verified against `np.dot` for correctness before benchmarking (Table 1).

5 Optimized Single-Kernel Matrix Multiplication

The second implementation removes the host-side double loop entirely by launching one thread per output element over a 2D grid of 2D thread blocks, and performs the full reduction over l inside the kernel, following Eq. (7) directly:

```
__global__ void customMatrixMul(float *A, float *B, float *C, int N) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    int col = blockIdx.y * blockDim.y + threadIdx.y;

    if (row < N && col < N) {
        float sum = 0.0f;
        for (int l = 0; l < N; ++l) {
            float a_val = A[l * N + row]; // A[i][l] -> A[l*N + i]
            float b_val = B[col * N + l]; // B[l][k] -> B[k*N + l]
            sum += a_val * b_val;
        }
        C[col * N + row] = sum;          // C[i][k] -> C[k*N + i]
    }
}
```

Each line of the comment block is a direct instance of the flattening rule (4), with $m = N$: `row` plays the role of i, `col` the role of k, and the loop variable `l` the role of the summation index. On the Python side, the host matrices are flattened in the same column-major convention before being copied to the device, and the result is reshaped back the same way:

```
A_flat = A.flatten(order='F').astype(np.float32)
B_flat = B.flatten(order='F').astype(np.float32)
...
custom_matmul_kernel(A_gpu, B_gpu, C_gpu, np.int32(N),
                     block=block_dim, grid=grid_dim)
cuda.memcpy_dtoh(C_flat, C_gpu)
return C_flat.reshape((N, N), order='F')
```

The thread grid is configured with 16×16 threads per block (256 threads, within the 1024-thread-per-block hardware limit) and the number of blocks is rounded up to cover the full $N \times N$ output:

$$\texttt{grid_dim} = \left(\left\lceil \frac{N}{16} \right\rceil, \left\lceil \frac{N}{16} \right\rceil\right). \tag{8}$$

The `if (row < N && col < N)` guard discards the extra threads that fall outside C when N is not a multiple of 16. Correctness was again verified against `np.dot` (here for $N = 20$) before benchmarking.

6 Benchmarks

Both implementations were timed against NumPy's `np.dot` (which itself calls an optimized BLAS routine on the CPU) over a range of matrix sizes, on the same T4 GPU.

6.1 Naive kernel-per-element approach

The naive approach is slower than the CPU at every tested size, and the gap widens with N: it issues N^2 separate kernel launches (one `dotProd` call per output element) plus an equal number of host↔device transfers, so per-launch and per-transfer overhead dominates completely over the tiny amount of actual arithmetic done per launch.

N	GPU time (s)	CPU time (s)
2	0.000477	0.000013
4	0.001763	0.000008
8	0.007404	0.000009
16	0.029942	0.000014
32	0.103170	0.000016
64	0.360964	0.000034
128	1.355726	0.007666

Table 1: Naive implementation (Section 4): GPU time grows roughly as $O(N^2)$ kernel launches, each with fixed overhead, so the GPU never becomes competitive in the tested range.

N	GPU time (s)	CPU time (s)
16	0.000203	0.000015
32	0.000755	0.000017
64	0.000268	0.000027
128	0.000338	0.000144
256	0.000870	0.000443
512	0.004370	0.002658
1024	0.026968	0.019651
2048	0.151141	0.156525
4096	1.015077	1.153762
8192	5.369192	10.768943

Table 2: Optimized implementation (Section 5): a single kernel launch per matrix product. The GPU overtakes the CPU between $N = 1024$ and $N = 2048$, and is roughly $2\times$ faster by $N = 8192$.

6.2 Optimized flattened kernel

Replacing N^2 tiny kernel launches with a single launch that performs the full $nm(2j - 1)$ operation count of Eq. (3) in parallel changes the qualitative behaviour entirely: at small N the fixed cost of a single kernel launch still dominates and the CPU wins, but the crossover occurs around $N \approx 2048$, after which the GPU's parallelism over N^2 threads outweighs the CPU's advantage, reaching roughly a factor of two speed-up by $N = 8192$.

7 Discussion

The two implementations isolate a single variable: the granularity at which work is handed to the GPU. The naive version parallelizes only the innermost reduction (length j) and keeps the $O(N^2)$ loop over output elements on the host; the optimized version parallelizes the $O(N^2)$ loop over output elements across the GPU's thread grid, with only the length-j reduction left sequential per thread. The benchmarks confirm that kernel-launch and transfer overhead, not raw FLOP throughput, is the dominant cost at these problem sizes, and that minimizing the number of host/device round-trips is the first-order optimization for a GPU port of this kind.

The current optimized kernel reads A and B directly from global memory inside the reduction loop, with no reuse across threads in the same block. A natural next step would be shared-memory tiling (loading sub-blocks of A and B into per-block shared memory once and reusing them across all threads in the block), which is the standard technique used by vendor libraries such as cuBLAS and typically yields a further significant speed-up over the N^2-thread, global-

memory-only version implemented here.

8 References

CUDA Guide